

Assignment #2

INSTRUCTIONS:

- EACH MEMBER **MUST** UNDERSTAND EVERYTHING IN THE ASSIGNMENT.
- ANY MEMBER MAY BE ASKED TO DO AN **UPDATE** OR **CHANGE** IN CODE.
- Assignment deadline will be **May 11 2026@ 11:45 PM**.
- Assignment discussions start in the week that starts **May 11, 2026**. [Discussion slots will be announced for each TA]
- Assignment's total grade is 10 marks. [Distribution is specified in assignment requirements below] + 1 Bonus mark.
- Late submissions will receive a deduction of 3 marks unless there is a clearly accepted reason sent by email immediately.
- **CHEATING** in the assignment is considered as **ZERO** from each member's total grade.
- In the discussion, all members **MUST** **present** in the discussion [Unless you have a clearly accepted reason sent by mail to the TA before the discussion], otherwise, there will be a grade deduction of 1 mark.

ASSIGNMENT REQUIREMENTS:

PART ONE:

- Work on the data in the JSON file attached named **mds.json**.
- Use the MongoDB container we worked on during the lab to access MongoDB, create a database with the name **moviesDB**, and import the JSON file data into a collection with the name **moviesColl**. **[0.5 Mark]**
- Each team must write MongoDB queries to answer the following questions:

a. Indexing Question:

Create an index on the **name** field, then write a query to search for a specific movie or TV show by its name. **[0.75 Mark]**

b. Logical Operations Question:

Write a query to find all titles where the **type** is **Movie** and the **genre** contains either **Drama** or **Documentary**. **[0.5 Mark]**

c. Aggregation Question:

Write an aggregation pipeline to count how many titles exist for each **type** such as **Movie** and **TV Show**, then sort the result from highest to lowest count. **[0.75 Mark]**

d. Query Question:

Write a query to find the top 5 highest-rated Movies that were released after 2010, are available on Prime Video or Netflix, and have an IMDb rating. The query should sort based on the numeric value of `imdb_rating`. The result should display only name, `released_at`, genre, `streaming_on`, country, and `imdb_rating`. **[0.5 Mark]**

For each query, the team should write a short comment describing the purpose of the query only.

Each team must submit screenshots showing:

- a. The MongoDB container running.
- b. The database **moviesDB** created.
- c. The collection **moviesColl** after importing the data.
- d. The command used to import the JSON file.
- e. The output of each query.

All required files must be submitted together as one ZIP file on Moodle.

PART TWO: (7 marks)

1. Convert the **mds.json** file from **JSON format** to **CSV format**.
2. Use the Hadoop container we used during the lab, and copy the converted CSV file to **HDFS**.
3. Write a MapReduce job using **Java in IntelliJ IDEA**. The instructions on how to create and run a MapReduce job in an IntelliJ project were explained in the lab and the slides.
4. The MapReduce job must calculate rating statistics for each combination of **Content Type, First Genre, and Release Decade**.
5. The program must be written in one class named **ARDriver.java**.
6. The class **ARDriver.java** must contain the **Driver, Mapper, and Reducer code**.
If the team completes the bonus task, the Combiner code must also be included inside **ARDriver.java**
7. The program must use only the **first genre** if more than one genre exists.
8. The program must extract the release decade from the **released_at** field.
9. The program must extract the numeric value from the **imdb_rating** field.
10. The program must exclude records that contain null, empty, or invalid values in genre, type, `released_at`, or `imdb_rating`.
11. The final output must display the **content type, genre, release decade, number of valid titles, average IMDb rating, minimum IMDb rating, and maximum IMDb rating**.

12. The MapReduce solution must be implemented using Java Hadoop MapReduce only. Spark, Python, Hive, or Pig are not allowed for the MapReduce job.

BONUS

- Write one additional MongoDB query using any advanced MongoDB feature not already used in Part One. **[0.5 Mark]**
- Use the concept of **Combiner** in the MapReduce job in Part Two to reduce intermediate data before sending it to the Reducer. **[0.5 Mark]**

PART TWO Marking Scheme:

Driver code: 1.5 Mark

Mapper code and data extraction: 2 Marks

Reducer code and statistics calculation: 2 Marks

Data cleaning and validation: 1.5 Marks

Total: 7 Marks

DELIVERABLES

Each team must submit **one ZIP file** on **Moodle**:

1. A text file named **mDBQ.txt** containing all MongoDB queries used in Part One.
2. A text file named **mDBR.txt** containing the results/output of the MongoDB queries.
3. The Java class file named **ARDriver.java** used for the MapReduce job.
4. The class ARDriver.java must contain the Driver, Mapper, and Reducer code. If the team completes the bonus task, the Combiner code must also be included inside ARDriver.java.
5. The output file generated from the MapReduce job named **part-r-00000**.
6. The converted CSV file used as input for the MapReduce job.
7. The ZIP file name must include the team number or the names/IDs of the team members.
8. Each team must make sure that all submitted files are complete and correctly named.
9. A PDF or Word report file containing all required screenshots.
10. Incomplete ZIP files may lose marks based on the missing files.

NOTE #1: JSON format to CSV format should include a header row with the field names.

Nested objects, if found, should be flattened using dot notation.

Array values, if found, should be written as comma-separated values inside one CSV cell.

Empty or null values should not be removed during the JSON to CSV conversion step.

The converted CSV file should keep the main dataset fields, such as name, released_at, genre, streaming_on, country, type, content_rating, imdb_rating, number_of_seasons, and cast_and_crew.

Data cleaning and validation should be handled inside the MapReduce job, not during the conversion step.

NOTE #2: The output of the MapReduce job should be like:

Type, Genre, Decade, Count, AverageRating, MinRating, MaxRating

Movie, Drama, 2010s, 25, 6.82, 4.2, 8.6

Movie, Horror, 2000s, 13, 7.15, 5.6, 8.9

TV Show, Drama, 2020s, 10, 7.60, 6.4, 8.5

TV Show, Comedy, 2010s, 8, 7.95, 6.9, 9.8

The program should use only the first genre if more than one genre exists.

The program should extract the release decade from the released_at field.

The program should extract the numeric rating value from the imdb_rating field.

Records with empty or invalid genre, type, released_at, or imdb_rating values should be excluded from the MapReduce calculation.

NOTE #3: If the team completes the Combiner bonus task, the Combiner code should be included inside the same Java file named ARDriver.java.

The Combiner should reduce intermediate data before sending it to the Reducer.